

Sistemas Embarcados - Prof. Douglas Renaux

Laboratório 3

Cássio Keisuke Yamauchi, Paulo Sérgio Ávila Júnior

¹Universidade Tecnológica Federal do Paraná – Campus Curitiba
Curitiba – PR – Brazil

{yamauchi, pauloj.2018}@alunos.utfpr.edu.br

1. Introdução

Este relatório é referente ao laboratório 3 da matéria de Sistemas Embarcados. O objetivo é implementar um algoritmo em Assembly e utilizá-lo como subrotina de um programa em C++, de modo a explorar o uso de rotinas em Assembly dentro de programas que utilizem C ou C++, evidenciando uma ferramenta útil na criação de programas de alta performance. Ainda que Assembly já não seja amplamente usado como era antes, é importante o entendimento dessa linguagem caso haja necessidade de retrocompatibilidade com programas mais antigos ou possibilidade de melhoria de performance se consertado uma "otimização" em Assembly indevida por parte do compilador.

2. Planejamento das fases do processo de desenvolvimento

As etapas que serão seguidas para a resolução deste Lab são as seguintes:

1. Planejamento do desenvolvimento da aplicação
2. Especificação do problema e da solução
3. Estudo da plataforma de HW
4. Projeto da solução
5. Identificação e estudo das ferramentas utilizadas na solução
6. Configuração do projeto na IDE
7. Desenvolvimento do código da solução
8. Teste e depuração
9. Entrega dos resultados

3. Especificação do problema

O problema a ser solucionado é a criação de um histograma (vetor de ocorrências) para os valores de pixel em uma imagem de no máximo 64kbytes em tamanho, usando de uma rotina escrita em assembly. Essa imagem é representada em escala de cinza por um vetor. O histograma será então emitido para a saída padrão do sistema por cout.

4. Especificação da solução

A solução elaborada deverá seguir os seguintes requisitos:

4.1. Requisitos Funcionais

- RF01: O programa deve gerar um histograma dos valores de pixel contidos em uma imagem em grayscale.
- RF02: O programa deve emitir os valores contidos no histograma pela saída padrão.

4.2. Requisitos Não-Funcionais

- RNF01: A rotina que gera o histograma deve ser escrita em Assembly ARM.
- RNF02: O programa deve ser escrito em C++.
- RNF03: Os valores devem ser emitidos utilizando do comando cout.
- RNF04: Os valores de pixel serão de 8-bits.
- RNF05: O tamanho máximo da imagem será de 65536 pixels.

5. Estudo da plataforma de HW

A placa Tiva TM4C1294NCPDT utiliza um processador Cortex M4F de 32-bits com conjunto de instruções Thumb-2 mistas de 16-/32-bit. Ele opera em 120MHz de clock máximo com 150 DMIPS, acesso desalinhado à memória e manipulação atômica de bits, além de 13 registradores de uso geral.

6. Projeto (design) da solução

Para atingir os objetivos especificados, será utilizado o fluxograma disponibilizado pelo professor, representado pela figura 1

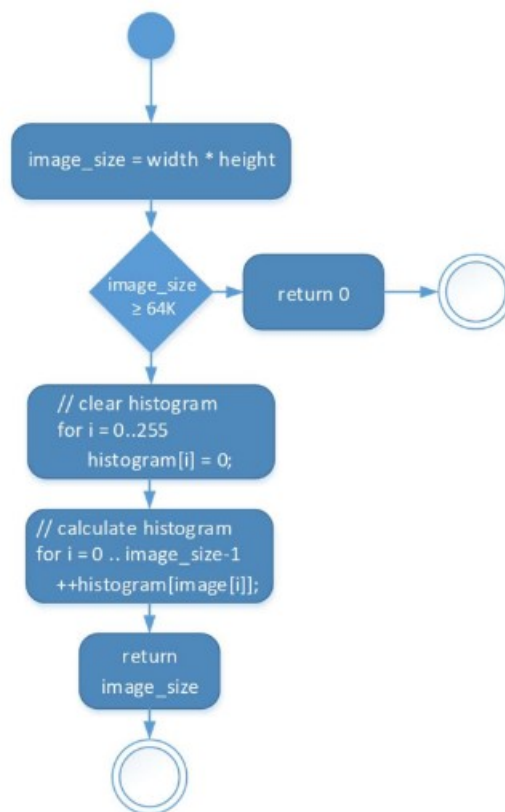


Figura 1. Fluxograma para criação do histograma

6.1. Planejamento das estruturas de dados

6.1.1. Representação da Imagem (input)

A imagem é representada por uma matriz em C. Então, é possível enxergar isso como um vetor contínuo em assembly.

6.1.2. Estruturas utilizadas para o processamento

Será utilizado um array para armazenar o histograma, em que cada posição no array representa um valor na escala de cinza (0-255) e cada número armazenado na posição representa a frequência desse determinado valor de pixel.

6.2. Forma de passagem de parâmetros

Os registradores utilizados para passagem de parâmetros são os seguintes:

- R0: Largura da imagem
- R1: Altura da imagem
- R2: Ponteiro para o início da imagem
- R3: Ponteiro para o histograma de retorno

O registrador R0 guardará o retorno da função ao retornar.

6.3. Algoritmo e alocação de variáveis aos registradores.

O algoritmo terá 5 partes, como demonstrado no fluxograma 1. O uso de registradores em cada parte está descrito a seguir.

1. Guardamos o tamanho da imagem `image_size` em R0.
2. A verificação do tamanho máximo da imagem será feita diretamente sobre o registrador R0, e caso o tamanho da imagem seja maior que 64K escrevemos 0 sobre o registrador R0 e retornamos.
3. Guardamos o iterador do loop `i` no registrador R4 e o utilizamos para iterar sobre o ponteiro do histograma, sem alterá-lo.
4. Reutilizamos o registrador R4 para `i` e iteramos diretamente os ponteiros em R2 e R3, sem alterá-los.
5. R5 é utilizado para ser o *offset*, indicando qual pixel do histograma deve ser incrementado.
6. Não precisamos fazer nada para retornar `image_size`, pois o valor já está em R0.

7. Configuração do projeto na IDE (IAR)

As configurações relevantes para o projeto Lab3 são as seguintes:

- Sem otimizações
- Compilador C++
- Target device: TM4C1294NCPDT
- Library Libc++
- Output executável

- Bibliotecas incluídas nas configurações de linker: Tivaware driverlib.a e Lab3_images.a

As configurações relevantes para o projeto Lab3_images são as seguintes:

- Sem otimizações
- Target device: TM4C1294NCPDT
- Output library
- Compilador standard C

8. Teste e depuração

É possível testar a validade do programa fazendo uma função-teste em C++ e comparando os resultados, utilizando a imagem0, conforme instruído pelo docente. Além disso, há um histograma da imagem disponibilizada pelo professor, sendo possível observar a similaridade entre o histograma criado e o real se utilizada a imagem1.